

```

int add(int, int);
int main()
{
    int a = 10, b = 20;
    printf("Sum of two numbers = %d\n", add(a, b)); // variables a, b are called actual arg
    return 0;
}

int add(int x, int y)
{
    // variables x, y are called formal parameters
    return(x + y);
}

```

In the above code whenever the function call `add(a, b)` is made, the execution control is transferred to the function definition of `add()`.

The values of actual arguments `a` and `b` are copied in to the formal arguments `x` and `y` respectively.

The formal parameters `x` and `y` are available only with in the function definition of `add()`. After completion of execution of `add()`, the control is transferred back to the `main()`.

See & retype the below code which will demonstrate about formal and actual arguments.

```
#include <stdio.h>
```

```
int sum(int);
```

```

int main()
{
    int number;
    scanf("%d", &number);
    printf("Sum of %d natural numbers = %d\n", number, sum(number));
    return 0;
}

```

```

int sum(int value)
{
    int i, total = 0;
    for (i = 1; i <= value; i++)
    {
        total = total + i;
    }
    return(total);
}

```

For example:

Input	Result
5	Sum of 5 natural numbers = 15

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int sum(int);
3 int main()
4 {
5     int number;
6     scanf("%d", &number);
7     printf("Sum of %d natural numbers = %d\n", number, sum(number));
8     return 0;
9 }
10 int sum(int value)
11 {
12     int i, total = 0;
13     for(i = 1; i <=value;i++)
14     {
15         total = total + i;
16     }
17     return (total);
18 }
19 }

```

	Input	Expected	Got	
✓	5	Sum of 5 natural numbers = 15	Sum of 5 natural numbers = 15	✓

Passed all tests! ✓

```

int main()
{
    int x = 10; // Local Variable x
    change();
    printf("%d", x); // The value 10 is printed
    return 0;
}

void change()
{
    printf("%d", x); // The value 20 is printed
}

```

In the above code the global and local variables have the same variable name x, but they are different.

In main() function the **local** variable x is only accessed, so it prints the value 10.

In change() function the variable x is not declared locally so it access **global** variable x, so it prints 20.

See & retype the below code which will demonstrate about local and global variables.

```

#include <stdio.h>

int x = 15;

void change1(int x)
{
    printf("In change1() function x = %d\n", x);
}

void change2()
{
    printf("In change2() function x = %d\n", x);
}

int main()
{
    int x = 10;
    printf("In main() function x = %d\n", x);
    change1(x);
    change2();
    printf("In main() function x = %d\n", x);
    return 0;
}

```

For example:

Result
In main() function x = 10
In change1() function x = 10
In change2() function x = 15
In main() function x = 10

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int x = 15;
3 void change1(int x)
4 {
5     printf("In change1() function x = %d\n",x);
6 }
7 void change2()
8 {
9     printf("In change2() function x = %d\n",x);
10 }
11 int main()
12 {
13     int x = 10;
14     printf("In main() function x = %d\n",x);
15     change1(x);
16     change2();
17     printf("In main() function x = %d\n",x);
18     return 0;
19 }
20

```

	Expected	Got	
✓	In main() function x = 10	In main() function x = 10	✓
	In change1() function x = 10	In change1() function x = 10	
	In change2() function x = 15	In change2() function x = 15	
	In main() function x = 10	In main() function x = 10	

Passed all tests! ✓

Global variables are declared outside of any function.

```

int main()
{
    int x = 10; // Local Variable x
    change();
    printf("%d", x); // The value 10 is printed
    return 0;
}

void change()
{
    printf("%d", x); // The value 20 is printed
}

```

In the above code the global and local variables have the same variable name x, but they are different.

In main() function the **local** variable x is only accessed, so it prints the value 10.

In change() function the variable x is not declared locally so it access **global** variable x, so it prints 20.

See & retype the below code which will demonstrate about local and global variables.

```

#include <stdio.h>

int x = 15;

void change1(int x)
{
    printf("In change1() function x = %d\n", x);
}

void change2()
{
    printf("In change2() function x = %d\n", x);
}

int main()
{
    int x = 10;
    printf("In main() function x = %d\n", x);
    change1(x);
    change2();
    printf("In main() function x = %d\n", x);
    return 0;
}

```

For example:

Result
In main() function x = 10
In change1() function x = 10
In change2() function x = 15
In main() function x = 10

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int x = 15;
3 void change1(int x)
4 {
5     printf("In change1() function x = %d\n",x);
6 }
7 void change2()
8 {
9     printf("In change2() function x = %d\n",x);
10 }
11 int main()
12 {
13     int x = 10;
14     printf("In main() function x = %d\n",x);
15     change1(x);
16     change2();
17     printf("In main() function x = %d\n",x);
18     return 0;
19 }
20

```

	Expected	Got	
✓	In main() function x = 10	In main() function x = 10	✓
	In change1() function x = 10	In change1() function x = 10	
	In change2() function x = 15	In change2() function x = 15	
	In main() function x = 10	In main() function x = 10	
Passed all tests! ✓			

accessed in every function.

Operating System calls the main() function at the time of execution. the variable a has no local declaration, so it access the global variable a.

In test() function also there is no local declaration of variable a, the variable a gets access from the global.

The global variables are destroyed only after completion of execution of entire program.

See & retype the below code which will demonstrate about global variables.

```
#include <stdio.h>

int a = 20;

void test();

int main()
{
    printf("In main() function a = %d\n", a);
    test();
    a = a + 15;
    printf("In main() function a = %d\n", a);
    return 0;
}

void test()
{
    a = a + 20;
    printf("In test() function a = %d\n", a);
}
```

For example:

Result
In main() function a = 20
In test() function a = 40
In main() function a = 55

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int a = 20;
3 void test();
4 int main()
5 {
6     printf("In main() function a = %d\n",a);
7     test ();
8     a = a + 15;
9     printf("In main() function a = %d\n",a);
10    return 0;
11 }
12 void test()
13 {
14     a = a+20;
15     printf("In test() function a = %d\n",a);
16 }
17
18
19
20
21
22
```

	Expected	Got	
✓	In main() function a = 20	In main() function a = 20	✓
	In test() function a = 40	In test() function a = 40	
	In main() function a = 55	In main() function a = 55	
Passed all tests! ✓			

A local variable is declared inside a function.

A **local variable** is visible only inside their function, only statements inside function can access that local variable.

Local variables are declared when the function execution started and local variables gets destroyed when control exits from function.

Let us consider an example:

```
#include <stdio.h>
void test();
int main()
{
    int a = 22, b = 44;
```

accessed in every function.

Operating System calls the `main()` function at the time of execution. the variable `a` has no local declaration, so it access the global variable `a`.

In `test()` function also there is no local declaration of variable `a`, the variable `a` gets access from the global.

The global variables are destroyed only after completion of execution of entire program.

See & retype the below code which will demonstrate about global variables.

```
#include <stdio.h>

int a = 20;

void test();

int main()
{
    printf("In main() function a = %d\n", a);
    test();
    a = a + 15;
    printf("In main() function a = %d\n", a);
    return 0;
}

void test()
{
    a = a + 20;
    printf("In test() function a = %d\n", a);
}
```

For example:

Result

```
In main() function a = 20
In test() function a = 40
In main() function a = 55
```

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int a = 20;
3 void test();
4 int main()
5 {
6     printf("In main() function a = %d\n",a);
7     test ();
8     a = a + 15;
9     printf("In main() function a = %d\n",a);
10    return 0;
11 }
12 void test()
13 {
14     a = a+20;
15     printf("In test() function a = %d\n",a);
16 }
17
18
19
20
21
22
```

	Expected	Got	
✓	In main() function a = 20 In test() function a = 40 In main() function a = 55	In main() function a = 20 In test() function a = 40 In main() function a = 55	✓

Passed all tests! ✓

A local variable is declared inside a function.

A **local variable** is visible only inside their function, only statements inside function can access that local variable.

Local variables are declared when the function execution started and local variables gets destroyed when control exits from function.

Let us consider an example:

```
#include <stdio.h>
void test();
int main()
{
    int a = 22, b = 44;
```

```

}

void test()
{
    int a = 50, b = 80;
    printf("Values in test() function a = %d and b = %d\n", a, b);
}

```

In the above code we have 2 functions main() and test(), in these functions local variables are declared with same variable names a and b but they are different.

Operating System calls the main() function at the time of execution. the **local variables** with in the main() are created when the main() starts execution.

when a call is made to test() function, first the control is transferred from main() to test(), next the local variables with in the test() are created and they are available only with in the test() function.

After completion of execution of test() function, the local variables are destroyed and the control is transferred back to the main() function.

See & retype the below code which will demonstrate about local variables.

```

#include <stdio.h>

void test();

int main()
{
    int a = 9, b = 99;
    test();
    printf("Values in main() function a = %d and b = %d\n", a, b);
    return 0;
}

void test()
{
    int a = 5, b = 55;
    printf("Values in test() function a = %d and b = %d\n", a, b);
}

```

For example:

Result
Values in test() function a = 5 and b = 55
Values in main() function a = 9 and b = 99

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 void test();
3 int main()
4 {
5     int a= 9,b= 99;
6     test();
7     printf("Values in main() function a = %d and b = %d\n",a,b);
8     return 0;
9 }
10 }
11 void test()
12 {
13     int a = 5, b = 55;
14     printf("Values in test() function a = %d and b = %d\n",a,b);
15 }

```

	Expected	Got	
✓	Values in test() function a = 5 and b = 55 Values in main() function a = 9 and b = 99	Values in test() function a = 5 and b = 55 Values in main() function a = 9 and b = 99	✓
Passed all tests! ✓			

Finish review



Saved

Started on	Sunday, 12 April 2026, 12:06 PM
State	Finished
Completed on	Sunday, 12 April 2026, 12:23 PM
Time taken	17 mins 33 secs
Name	PAVITHRA B 2025-BME

Question 1
Correct
Marked out of 1.00
Flag question

Fill the missing code to understand the concept of a function with arguments and without return value.

Note: Take pi value as 3.14

The below code is to find the area of circle using functions.

For example:

Input	Result
11.23	Area of circle = 395.994476

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 void area_circle(float);
4
5 int main()
6 {
7     float radius;
8     scanf("%f", &radius);
9     area_circle(radius);
10    return 0;
11 }
12
13 void area_circle(float r)
14 {
15     //Correct the code
16     float area;
17     area = 3.14*r*r;
18     // Write the code to calculate the area of circle
19     printf("Area of circle = %f\n", area);
20 }
```

	Input	Expected	Got	
✓	11.23	Area of circle = 395.994476	Area of circle = 395.994476	✓

Passed all tests! ✓

Question 2
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below code to understand about function with arguments and with return value.

The below code is to find the factorial of a given number using functions.

For example:

Input	Result
3	Factorial of a given number 3 = 6

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int factorial(int);
4
5 int main()
6 {
7     int number;
8     scanf("%d", &number);
9     printf("Factorial of a given number %d = %d\n", number, factorial(number));
10    return 0;
11 }
12
13 int factorial(int n)
14 {
15     int i, fact = 1;
16     for (i = 1; i <= n; i++)
17     {
18         fact = fact*i; // Write code to calculate the factorial of a given num
19     }
20    return fact;
21    // Write the return statement
22 }
```

✓	3	Factorial of a given number 3 = 6	Factorial of a given number 3 = 6	✓
---	---	-----------------------------------	-----------------------------------	---

Passed all tests! ✓

When a **function definition** has arguments, it receives data from the calling function.

After taking some desired action, only one value will be returned from **called function** to **calling function** through the return statement.

If a function returns a value, the **function call** may appear in any expression and the returned value used as an operand in the evaluation of the expression.

Let us consider an example of a function with arguments and with return value:

```
#include <stdio.h>
int largest(int, int, int);
int main()
{
    int a, b, c;
    printf("Enter three numbers : ");
    scanf("%d%d%d", &a, &b, &c);
    printf("Largest of the given three numbers = %d\n", largest(a, b, c));
    return 0;
}
int largest(int x, int y, int z)
{
    if ((x > y) && (x > z))
    {
        return x;
    }
    else if (y > z)
    {
        return y;
    }
    else
    {
        return z;
    }
}
```

In the above sample code the function int largest(int, int, int); specifies that the function receives three values and returns a value to the **calling function**.

Fill in the missing code in the below program to find the largest of three numbers using **largest()** function.

For example:

Input	Result
99 49 29	Largest of the given three numbers = 99
45 67 35	Largest of the given three numbers = 67

Answer: (penalty regime: 0 %)

Reset answer

```
1
2
3 int largest(int, int, int);
4
5 int main()
6 {
7     int a, b, c;
8     scanf("%d%d%d", &a, &b, &c);
9     printf("Largest of the given three numbers = %d\n", largest(a,b,c)); // C
10    return 0;
11 }
12
13 int largest(int x,int y,int z)
14 {
15     // Correct the code
16     if (x>y && x>z)
17     {
18         // Correct the code
19         return x ; // Correct the code
20     }
21     else if (y>z)
22     {
```

	Input	Expected	Got
✓	99 49 29	Largest of the given three numbers = 99	Largest of the given three numbers = 99
✓	45 67 35	Largest of the given three numbers = 67	Largest of the given three numbers = 67

Passed all tests! ✓

When a function has **no arguments**, it does not receive any data from the calling function.

When a function **return a value**, the calling function receives data from the called function.

Let us consider an example of a function without arguments and without return value.

Question 3
 Correct
 Marked out of 1.00
 Flag question

When a **function definition** has arguments, it receives data from the calling function.

After taking some desired action, only one value will be returned from **called function** to **calling function** through the return statement.

If a function returns a value, the **function call** may appear in any expression and the returned value used as an operand in the evaluation of the expression.

Let us consider an example of a function with arguments and with return value:

```
#include <stdio.h>
int largest(int, int, int);
int main()
{
    int a, b, c;
    printf("Enter three numbers : ");
    scanf("%d%d%d", &a, &b, &c);
    printf(" Largest of the given three numbers = %d\n", largest(a, b, c));
    return 0;
}
int largest(int x, int y, int z)
{
    if ((x > y) && (x > z))
    {
        return x;
    }
    else if (y > z)
    {
        return y;
    }
    else
    {
        return z;
    }
}
```

In the above sample code the function int largest(int, int, int); specifies that the function receives three values and returns a value to the **calling function**.

Fill in the missing code in the below program to find the largest of three numbers using **largest()** function.

For example:

Input	Result
99 49 29	Largest of the given three numbers = 99
45 67 35	Largest of the given three numbers = 67

Answer: (penalty regime: 0 %)

Reset answer

```
1 // Correct the code
2
3 int largest(int, int, int);
4
5 int main()
6 {
7     int a, b, c;
8     scanf("%d%d%d", &a, &b, &c);
9     printf("Largest of the given three numbers = %d\n", largest(a,b,c)); // Correct the code
10    return 0;
11 }
12
13 int largest(int x,int y,int z)
14 {
15     // Correct the code
16     if (x>y && x>z)
17     {
18         // Correct the code
19         return x ; // Correct the code
20     }
21     else if (y>z)
22     {
23         // Correct the code
24         return y ; // Correct the code
25     }
26     else
27     {
28         return z;
29     }
30 }
```

	Input	Expected	Got
✓	99 49 29	Largest of the given three numbers = 99	Largest of the given three numbers = 99
✓	45 67 35	Largest of the given three numbers = 67	Largest of the given three numbers = 67

Passed all tests! ✓

Question 4
 Correct
 Marked out of 1.00
 Flag question

When a function has **no arguments**, it does not receive any data from the calling function.

When a function **return a value**, the calling function receives data from the called function.

Let us consider an example of a function without arguments and with return value:

When a function has **no arguments**, it does not receive any data from the calling function.

When a function **return a value**, the calling function receives data from the called function.

Let us consider an example of a function without arguments and with return value:

```
#include <stdio.h>
int sum(void);
int main()
{
    printf("\nSum of two given values = %d\n", sum());
    return 0;
}
int sum() {
    int a, b, total;
    printf("Enter two numbers : ");
    scanf("%d %d", &a, &b);
    total = a + b;
    return total;
}
```

In the above sample code the function `int sum(void);` specifies that the function does not receive any arguments but return a value to the **calling function**.

Fill in the missing code in the below program to find sum of two integers.

For example:

Input	Result
9 5	Sum of two given values = 14
45 78	Sum of two given values = 123

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int sum(void);
4
5 int main()
6 {
7     printf("Sum of two given values = %d\n", sum());
8     return 0;
9 }
10
11 int sum()
12 {
13     int a,b,total;
14     scanf("%d %d",&a,&b);
15     total = a+b;
16     return total; // Fill in the missing code
17     // Read two integers
18     // Find sum
19     // Return sum
20 }
```

	Input	Expected	Got	
✓	9 5	Sum of two given values = 14	Sum of two given values = 14	✓
✓	45 78	Sum of two given values = 123	Sum of two given values = 123	✓

Passed all tests! ✓

Write a **C** program to demonstrate functions without arguments and with return value.

The below code is used to check whether the given number is a prime number or not.

Write the function **prime()**.

Sample Input and Output:

5
The given number is a prime number

For example:

Input	Result
5	The given number is a prime number
27	The given number is not a prime number
121	The given number is not a prime number

Question 4

Correct

Marked out of
1.00

Flag question

When a function has **no arguments**, it does not receive any data from the calling function.

When a function **return a value**, the calling function receives data from the called function.

Let us consider an example of a function without arguments and with return value:

```
#include <stdio.h>
int sum(void);
int main()
{
    printf("\nSum of two given values = %d\n", sum());
    return 0;
}
int sum() {
    int a, b, total;
    printf("Enter two numbers : ");
    scanf("%d%d", &a, &b);
    total = a + b;
    return total;
}
```

In the above sample code the function `int sum(void);` specifies that the function does not receive any arguments but return a value to the **calling function**.

Fill in the missing code in the below program to find sum of two integers.

For example:

Input	Result
9 5	Sum of two given values = 14
45 78	Sum of two given values = 123

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int sum(void);
4
5 int main()
6 {
7     printf("Sum of two given values = %d\n", sum());
8     return 0;
9 }
10
11 int sum()
12 {
13     int a,b,total;
14     scanf("%d %d",&a,&b);
15     total = a+b;
16     return total; // Fill in the missing code
17     // Read two integers
18     // Find sum
19     // Return sum
20 }
```

	Input	Expected	Got	
✓	9 5	Sum of two given values = 14	Sum of two given values = 14	✓
✓	45 78	Sum of two given values = 123	Sum of two given values = 123	✓

Passed all tests! ✓

Question 5

Correct

Marked out of
1.00

Flag question

Write a **C** program to demonstrate functions without arguments and with return value.

The below code is used to check whether the given number is a prime number or not.

Write the function **prime()**.

Sample Input and Output:

5
The given number is a prime number

For example:

Input	Result
5	The given number is a prime number
27	The given number is not a prime number
121	The given number is not a prime number

✓	45 78	Sum of two given values = 123	Sum of two given values = 123	✓
---	-------	-------------------------------	-------------------------------	---

Passed all tests! ✓

Write a C program to demonstrate functions without arguments and with return value.

The below code is used to check whether the given number is a prime number or not.

Write the function `prime()`.

Sample Input and Output:

5

The given number is a prime number.

For example:

Input	Result
5	The given number is a prime number
27	The given number is not a prime number
121	The given number is not a prime number
1	The given number is not a prime number

Answer: (penalty regime: 0 %)

Reset answer

```

25 +     if(n%1==0){
26 +         count++;
27 +         break;
28 +     }
29 + }
30 + if(count ==0)
31 + {
32 +     return 0;
33 + }
34 + else
35 + {
36 +     return 1;
37 + }
38 + }
39 +
40 +
41 +
42 +
43 +
44 +
45 + // Write the function prime()
46 +

```

	Input	Expected	Got	
✓	5	The given number is a prime number	The given number is a prime number	✓
✓	27	The given number is not a prime number	The given number is not a prime number	✓
✓	121	The given number is not a prime number	The given number is not a prime number	✓
✓	1	The given number is not a prime number	The given number is not a prime number	✓

Passed all tests! ✓

Write a C program to demonstrate functions without arguments and without return value.

Write the functions `print()` and `hello()`.

The output is:

```

...***...
Hello! REC
...***...

```

For example:

Result
...***... Hello! REC ...***...

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2

```

n Sunday, 12 April 2026, 12:06 PM
e Finished
n Sunday, 12 April 2026, 12:23 PM
n 17 mins 33 secs
e PAVITHRA B 2025-BME

Fill the missing code to understand the concept of a function with arguments and without return value.

Note: Take pi value as 3.14

The below code is to find the area of circle using functions.

For example:

Input	Result
11.23	Area of circle = 395.994476

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 void area_circle(float);
4
5 int main()
6 {
7     float radius;
8     scanf("%f", &radius);
9     area_circle(radius);
10    return 0;
11 }
12
13 void area_circle(float r)
14 {
15     //Correct the code
16     float area;
17     area = 3.14*r*r;
18     // Write the code to calculate the area of circle
19     printf("Area of circle = %f\n", area);
20 }
```

	Input	Expected	Got	
✓	11.23	Area of circle = 395.994476	Area of circle = 395.994476	✓

Passed all tests! ✓

Fill in the missing code in the below code to understand about function with arguments and with return value.

The below code is to find the factorial of a given number using functions.

For example:

Input	Result
3	Factorial of a given number 3 = 6

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int factorial(int);
4
5 int main()
6 {
7     int number;
8     scanf("%d", &number);
9     printf("Factorial of a given number %d = %d\n", number, factorial(number));
10    return 0;
11 }
12
13 int factorial(int n)
14 {
15     int i, fact = 1;
16     for (i = 1; i <= n; i++)
17     {
18         fact = fact*i; // Write code to calculate the factorial of a given num
19     }
20     return fact;
21     // Write the return statement
22 }
```

```

void test()
{
    int a = 50, b = 80;
    printf("Values in test() function a = %d and b = %d\n", a, b);
}

```

In the above code we have 2 functions main() and test(), in these functions local variables are declared with same variable names a and b but they are different.

Operating System calls the main() function at the time of execution. the **local variables** with in the main() are created when the main() starts execution.

when a call is made to test() function, first the control is transferred from main() to test(), next the local variables with in the test() are created and they are available only with in the test() function.

After completion of execution of test() function, the local variables are destroyed and the control is transferred back to the main() function.

See & retype the below code which will demonstrate about local variables.

```

#include <stdio.h>

void test();

int main()
{
    int a = 9, b = 99;
    test();
    printf("Values in main() function a = %d and b = %d\n", a, b);
    return 0;
}

void test()
{
    int a = 5, b = 55;
    printf("Values in test() function a = %d and b = %d\n", a, b);
}

```

For example:

Result
Values in test() function a = 5 and b = 55
Values in main() function a = 9 and b = 99

Answer: (penalty regime: 0 %)

```

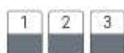
1 #include <stdio.h>
2 void test();
3 int main()
4 {
5     int a = 9, b = 99;
6     test();
7     printf("Values in main() function a = %d and b = %d\n", a, b);
8     return 0;
9 }
10 }
11 void test()
12 {
13     int a = 5, b = 55;
14     printf("Values in test() function a = %d and b = %d\n", a, b);
15 }

```

Expected	Got
✓ Values in test() function a = 5 and b = 55 Values in main() function a = 9 and b = 99	Values in test() function a = 5 and b = 55 ✓ Values in main() function a = 9 and b = 99
Passed all tests! ✓	

Finish review

Quiz navigation



Show one page at a time

Finish review



Saved

✓ 45.78 Sum of two given values = 123 Sum of two given values = 123 ✓

Passed all tests! ✓

Question 5

Correct

Marked out of 1.00

Flag question

Write a C program to demonstrate functions without arguments and with return value.

The below code is used to check whether the given number is a prime number or not.

Write the function **prime()**.

Sample Input and Output:

5

The given number is a prime number

For example:

Input	Result
5	The given number is a prime number
27	The given number is not a prime number
121	The given number is not a prime number
1	The given number is not a prime number

Answer: (penalty regime: 0 %)

Reset answer

```
25  if(n%1==0){
26      count++;
27      break;
28  }
29  }
30  if(count ==0)
31  {
32      return 0;
33  }
34  else
35  {
36      return 1;
37  }
38  }
39
40
41
42
43
44
45  // Write the function prime()
46
```

	Input	Expected	Got	
✓	5	The given number is a prime number	The given number is a prime number.	✓
✓	27	The given number is not a prime number	The given number is not a prime number	✓
✓	121	The given number is not a prime number	The given number is not a prime number	✓
✓	1	The given number is not a prime number	The given number is not a prime number	✓

Passed all tests! ✓

Question 6

Correct

Marked out of 1.00

Flag question

Write a C program to demonstrate functions without arguments and without return value.

Write the functions **print()** and **hello()**.

The output is:

```
...***...
Hello! REC
...***...
```

For example:

Result
...***... Hello! REC ...***...

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3
```

	Input	Expected	Got	
✓	5	The given number is a prime number	The given number is a prime number	✓
✓	27	The given number is not a prime number	The given number is not a prime number	✓
✓	121	The given number is not a prime number	The given number is not a prime number	✓
✓	1	The given number is not a prime number	The given number is not a prime number	✓

Passed all tests! ✓

Write a C program to demonstrate functions without arguments and without return value.

Write the functions `print()` and `hello()`.

The output is:

```
...***...
Hello! REC
...***...
```

For example:

Result

```
...***...
Hello! REC
...***...
```

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 void print()
4 {
5     printf("...***...\n");
6 } // Write the functions
7 void hello()
8 {
9     printf("Hello! REC\n");
10 }
11 int main()
12 {
13     print();
14     hello();
15     print();
16     return 0;
17 }
```

	Expected	Got	
✓	...***... Hello! REC ...***...	...***... Hello! REC ...***...	✓

Passed all tests! ✓

When a function definition has **arguments**, it receives data from the calling function.

The **actual arguments** in the function call must correspond to the **formal parameters** in the function definition, i.e. the number of actual arguments must be the same as the number of formal parameters, and each actual argument must be of the same data type as its corresponding formal parameter.

The **formal parameters** must be valid variable names in the function definition and the **actual arguments** may be variable names, expressions or constants.

The variables used in actual arguments are evaluated before the **function call** is made. When a function call is made, copies of the actual arguments are passed to the **called function**.

What occurs inside the function will have no effect on the variables used in the **actual argument** list. There may be several different calls to the same function from various places with a program.

Let us consider an example of a function with arguments and without return value:



Saved

	Input	Expected	Got	
✓	5	The given number is a prime number	The given number is a prime number	✓
✓	27	The given number is not a prime number	The given number is not a prime number	✓
✓	121	The given number is not a prime number	The given number is not a prime number	✓
✓	1	The given number is not a prime number	The given number is not a prime number	✓

Passed all tests! ✓

Question 6

Correct

Marked out of 1.00

Flag question

Write a C program to demonstrate functions without arguments and without return value.

Write the functions `print()` and `hello()`.

The output is:

```
...***...
Hello! REC
...***...
```

For example:

Result

```
...***...
Hello! REC
...***...
```

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 void print()
4 {
5     printf("...***...\n");
6 } // Write the functions
7 void hello()
8 {
9     printf("Hello! REC\n");
10 }
11 int main()
12 {
13     print();
14     hello();
15     print();
16     return 0;
17 }
```

	Expected	Got	
✓	...***... Hello! REC ...***...	...***... Hello! REC ...***...	✓

Passed all tests! ✓

Question 7

Correct

Marked out of 1.00

Flag question

When a function definition has **arguments**, it receives data from the calling function.

The **actual arguments** in the function call must correspond to the **formal parameters** in the function definition, i.e. the number of actual arguments must be the same as the number of formal parameters, and each actual argument must be of the same data type as its corresponding formal parameter.

The **formal parameters** must be valid variable names in the function definition and the **actual arguments** may be variable names, expressions or constants.

The variables used in actual arguments are copied into the formal parameters before the function call is made. When a function call is made, copies of the actual arguments are passed to the called function.

What occurs inside the function will have no effect on the variables used in the **actual argument** list. There may be several different calls to the same function from various places with a program.

Let us consider an example of a function with arguments and without return value:



Saved

i.e. the number of actual arguments must be the same as the number of formal parameters, and each actual argument must be of the same data type as its corresponding formal parameter.

The **formal parameters** must be valid variable names in the function definition and the **actual arguments** may be variable names, expressions or constants in the function call.

The variables used in actual arguments must be assigned values before the **function call** is made. When a function call is made, copies of the values of actual arguments are passed to the **called function**.

What occurs inside the function will have no effect on the variables used in the **actual argument** list. There may be several different calls to the same function from various places with a program.

Let us consider an example of a function with arguments and without return value:

```
#include <stdio.h>
void largest(int, int);
int main()
{
    int a, b;
    printf("Enter two numbers : ");
    scanf("%d%d", &a, &b);
    largest(a, b);
    return 0;
}
void largest(int x, int y)
{
    if (x > y)
    {
        printf("Largest element = %d\n", x);
    }
    else
    {
        printf("Largest element = %d\n", y);
    }
}
```

In the above sample code the function void largest(int, int); specifies that the function receives two integer arguments from the **calling function** and does not return any value to the **called function**.

When the function call largest(a, b) is made in the main() function, the values of actual arguments a and b are copied in to the formal parameters x and y.

After completion of execution of largest(int x, int y) function, it does not return any value to the main() function. Simply the control is transferred to the main() function.

Fill in the missing code in the below program to find the largest of two numbers using **largest()** function.

For example:

Input	Result
27 18	Largest element = 27
13 17	Largest element = 17

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 void largest(int x, int y);
4
5 int main()
6 {
7     int a, b;
8     scanf("%d%d", &a, &b);
9     largest(a,b); // Correct the code
10    return 0;
11 }
12
13 void largest(int x,int y)
14 {
15     // Correct the code
16     if (x>y)
17     {
18         // Correct the code
19         printf("Largest element = %d\n", x);
20     }
21     else
22     {
```

	Input	Expected	Got	
✓	27 18	Largest element = 27	Largest element = 27	✓
✓	13 17	Largest element = 17	Largest element = 17	✓

Passed all tests! ✓

Argument must be of the same data type as its corresponding formal parameter.

The **formal parameters** must be valid variable names in the function definition and the **actual arguments** may be variable names, expressions or constants in the function call.

The variables used in actual arguments must be assigned values before the **function call** is made. When a function call is made, copies of the values of actual arguments are passed to the **called function**.

What occurs inside the function will have no effect on the variables used in the **actual argument** list. There may be several different calls to the same function from various places with a program.

Let us consider an example of a function with arguments and without return value:

```
#include <stdio.h>
void largest(int, int);
int main()
{
    int a, b;
    printf("Enter two numbers : ");
    scanf("%d%d", &a, &b);
    largest(a, b);
    return 0;
}
void largest(int x, int y)
{
    if (x > y)
    {
        printf("Largest element = %d\n", x);
    }
    else
    {
        printf("Largest element = %d\n", y);
    }
}
```

In the above sample code the function void largest(int, int); specifies that the function receives two integer arguments from the **calling function** and does not return any value to the **called function**.

When the function call largest(a, b) is made in the main() function, the values of actual arguments a and b are copied in to the formal parameters x and y.

After completion of execution of largest(int x, int y) function, it does not return any value to the main() function. Simply the control is transferred to the main() function.

Fill in the missing code in the below program to find the largest of two numbers using **largest()** function.

For example:

Input	Result
27 18	Largest element = 27
13 17	Largest element = 17

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 void largest(int x, int y);
4
5 int main()
6 {
7     int a, b;
8     scanf("%d%d", &a, &b);
9     largest(a,b); // Correct the code
10    return 0;
11 }
12
13 void largest(int x,int y)
14 {
15     // Correct the code
16     if (x>y)
17     {
18         // Correct the code
19         printf("Largest element = %d\n", x);
20     }
21     else
22     {
```

	Input	Expected	Got	
✓	27 18	Largest element = 27	Largest element = 27	✓
✓	13 17	Largest element = 17	Largest element = 17	✓

Passed all tests! ✓

All the **C** functions can be called either with **arguments** or without arguments in a C program. These functions may or may not **return values** to the calling function.

Depending on the **arguments** and **return values** functions are classified into 4 categories.

1. Function without arguments and without return value
2. Function with arguments and without return value
3. Function without arguments and with return value
4. Function with arguments and with return value

When a function has **no arguments**, it does not receive any data from the calling function.

Similarly, when a function **does not return a value**, the calling function does not receive any data from the called function.

In effect, there is no data transfer between the calling function and the called function in the category **function without arguments and without return value**.

Let us consider an example of a function without arguments and without return value:

```
#include <stdio.h>
void india_capital(void);
int main()
{
    india_capital();
    return 0;
}
void india_capital()
{
    printf("New Delhi is the capital of India\n");
}
```

In the above sample code the function void india_capital(void); specifies that the function does not receive any arguments and does not return any value to the main() function.

Identify the below errors and correct them.

For example:

Result

New Delhi is the capital of India

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 void india_capital();
4
5 int main()
6 {
7     india_capital();
8     return 0;
9 }
10
11 void india_capital()
12 {
13     printf("New Delhi is the capital of India\n");
14 }
```

Expected	Got	
✓ New Delhi is the capital of India	New Delhi is the capital of India	✓

Passed all tests! ✓

Finish review